

XTrimFitGym System Review

Historical baseline: this document records the system before remediation. The implemented changes, verification results, and remaining operator actions are tracked in `REVISION_REPORT.md`.

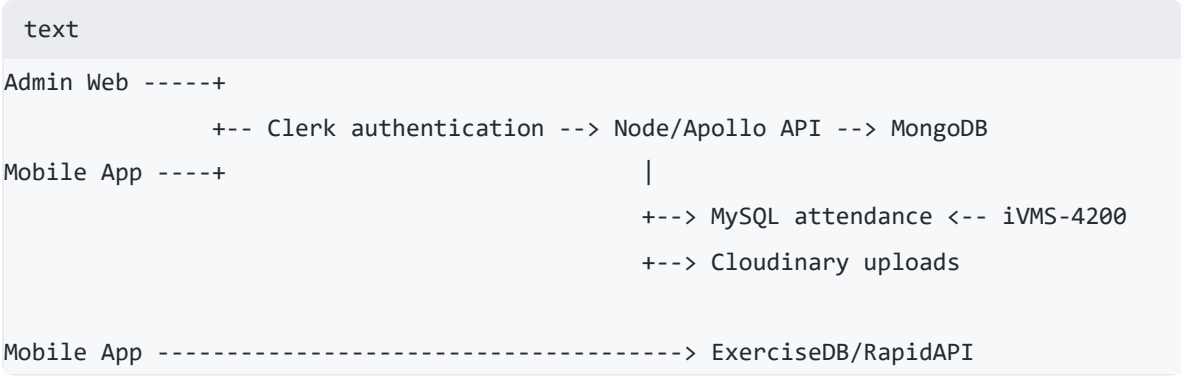
Review date: October 8, 2026
Scope: `Api` , `Mobile App` , and `Web`
Review type: Original read-only static architecture, integration, security, and maintainability review

Executive summary

The system has a coherent prototype architecture, but the current snapshot should not be considered production-safe. The largest concerns are authorization gaps, membership-state inconsistencies, exposed configuration, an insecure legacy attendance database, and multiple stale copies of the API and GraphQL schema.

No implementation changes were made during the review.

Current system map



The intended responsibilities appear to be:

- `Web` : administrator portal.
- `Mobile App` : member and coach application.
- `Api` : authentication mapping, GraphQL, uploads, business logic, notifications, and attendance integration.
- MongoDB: primary application database.
- MySQL: external biometric attendance bridge.

Highest-priority findings

Priority	Finding	Impact
Critical	Members can alter protected membership data through <code>updateUser</code>	Membership/payment and biometric onboarding can be bypassed
Critical	Development verification/sign-in operations exist in the production GraphQL schema	Anyone with API/log access could obtain or abuse verification codes
Critical	A RapidAPI credential is committed into the mobile configuration	The key can be extracted from source or a built app and abused
Critical	MySQL 5.5 is publicly exposed without TLS and grants the iVMS user full database privileges	High compromise and data-loss risk
High	Membership status has multiple conflicting sources of truth	Expired or canceled members may retain application access
High	Resolver authorization is inconsistent	Coaches or authenticated users can read data they do not own
High	Generated GraphQL clients cannot currently be reproduced	Future builds can silently use incompatible contracts
High	Three different API source snapshots are present	It is unclear which implementation is authoritative
High	Almost no meaningful automated tests or complete CI exist	Core regressions will only be found manually

1. Members can self-modify protected state

The generic `updateUser` mutation allows a user to update their own profile, but it also accepts and writes:

- `membershipId`
- `coachesIds`
- `facilityBiometricEnrollmentComplete`

See `Api/src/graphql/user/user-resolvers.ts` , around lines 546 and 649.

A member could therefore assign themselves a plan, alter coach relationships, or declare biometric enrollment complete. These fields need administrator-only or dedicated workflow

mutations.

2. Development authentication is exposed

The API schema includes:

- `requestDevEmailVerificationCode`
- `requestDevCoachSignInCode`
- `devCoachSignIn`

The codes are generated with `Math.random`, stored in memory, and written to logs. They are not protected by a production-environment check. See `Api/src/graphql/user/user-resolvers.ts`, around line 887.

These operations should not be reachable in production. The legacy JWT tokens created here and elsewhere also have no token expiry; only the browser cookie expires.

3. Authorization is inconsistent

Concrete examples:

- `getCoachRequests(coachId)` only checks whether the caller is logged in. Any authenticated user can request another coach's populated requests. See `Api/src/graphql/coachRequest/coachRequest-resolvers.ts`, around line 46.
- Any user with the `coach` role can query progress records for any member; the resolver does not verify the coach/member relationship. See `Api/src/graphql/progress/progress-resolvers.ts`, around line 165.
- Active-membership validation during session creation runs only when an administrator schedules the session, not when a coach does. See `Api/src/graphql/session/session-resolvers.ts`, around line 942.

Authorization is implemented manually in every resolver. That approach is already producing gaps and should be replaced by reusable policy functions.

4. Attendance infrastructure is unsafe

The attendance container uses MySQL 5.5, disables SSL, exposes MySQL through Railway's public TCP proxy, and grants the iVMS account all privileges on the database.

Evidence:

- `Api/railway-mysql/Dockerfile`
- `Api/railway-mysql/custom.cnf`
- `Api/railway-mysql/02-init-user.sh`

If iVMS genuinely requires this old protocol, it should be isolated behind a private network or ingestion gateway. The API should not share the same fully privileged public account.

Migrations 04 and 05 are not copied into Docker's initialization directory, so database correctness depends on someone remembering to execute them manually.

Disconnected or conflicting functionality

Membership state

There are at least three membership indicators:

1. `User.membershipDetails.membership_id`
2. `MembershipTransaction.status`
3. `MembershipTransaction.expiresAt`

The mobile app considers a member active solely when a membership ID exists. See `Mobile App/utils/memberMembership.ts`.

The API marks expired transactions lazily when `getCurrentMembership` is called, but it does not remove the membership ID when expiry is detected. Cancellation does remove it. Consequently, expiry and cancellation behave differently.

The mobile synchronization component stops polling once a membership ID appears. See `Mobile App/components/MemberMeSyncAndWelcome.tsx`. Its normal synchronization mechanism therefore will not discover later expiry or cancellation.

There are also two competing purchase workflows:

- `purchaseMembership` immediately creates an active transaction.
- `createSubscriptionRequest` requires administrator approval.

Both remain exposed in the schema and generated clients. A single authoritative subscription workflow needs to be chosen.

Notifications

The API creates persistent inactivity, expiry, and scheduled-session notifications.

However:

- Mobile does not query `getMyNotifications`. Its drawer reconstructs notifications from several unrelated queries and local storage.
- Web has a real system-notification component, but it is disabled unless `VITE_ENABLE_SYSTEM_NOTIFICATIONS=true`. See `Web/src/components/SystemNotificationBell.tsx`.
- That variable is absent from the Web environment example.

The API notification automation is therefore mostly invisible to users.

GraphQL contracts

The merged layout broke both client generation paths:

- Mobile expects `../XTrimFitGym-API/...` . See `Mobile App/codegen.ts` .
- Web expects `./api/XTrimFitGym-API-clean/...` . See `Web/codegen.ts` .

Neither points to the actual `Api/src/graphql` directory.

There is already proof of generated-code drift: the active API schema contains `SESSION_SCHEDULED` , but the API's generated resolver types omit it. Compare `Api/src/graphql/notification/notification-typeDefs.graphql` with `Api/src/types/types.ts` .

`MemberDetails.membershipTransaction` is declared in the schema, but no resolver populates it, so that field will normally resolve to `null` . `User.currentMembership` is the implemented version.

Authentication

The system currently mixes:

- Clerk sessions
- Legacy JWT bearer tokens
- Legacy authentication cookies
- Persisted Redux authentication state

Web saves the Clerk bearer token directly in `localStorage` , both explicitly and through persisted Redux. See `Web/src/store/slices/authSlice.ts` . That increases the impact of any XSS issue.

The WebSocket client connects immediately with `lazy: false` , before the Clerk token getter is registered. See `Web/src/lib/apollo/client.ts` . A first-time session can therefore establish an unauthenticated subscription connection and may not reconnect with credentials until later.

Mobile uses `SecureStore` for Clerk's token cache, but also retains legacy authentication and user data in `AsyncStorage`. Route protection is largely based on persisted Redux user state rather than Clerk's current loaded/signed-in state.

Repository organization

This is currently three folders placed together rather than a functioning monorepo:

- No root manifest or workspace configuration.
- No root README or system documentation before this review.
- No unified commands for install, code generation, checks, or development.

- No common environment-variable schema.
- No root CI.
- No Git history is visible in the reviewed copy.

There are several stale source copies:

- `Api/XTrimFitGym-API-clean`
- `Mobile App/XTrimFitGym-API-main`
- Active `Api/src`

The two old API copies have materially diverged from the active API. They should not remain beside production source because developers and tooling can easily select the wrong one.

Web also has two byte-identical GraphQL trees:

- `Web/graphql`
- `Web/src/graphql`

The API `dist` directory contains compiled API code, stale artifacts, and an entire Web build. Because the API build does not clean `dist` first, deleted source files can survive into later deployments. Its build script also executes `npm install` internally, making builds less reproducible. See `Api/package.json`.

The API's `vercel.json` appears copied from an older Web layout and points to a nonexistent `WEB ADMIN` directory.

Folder-level assessment

API

Positive foundations include a reasonably clear domain split, Mongo indexes for several important models, disabled-account checks, an explicit CORS allowlist, and documented attendance integration.

Main weaknesses:

- No automated tests.
- No health/readiness endpoints.
- No startup validation for required environment variables.
- No graceful shutdown handling.
- No rate limiting, GraphQL complexity controls, or general input-validation layer.
- In-memory GraphQL PubSub and scheduled jobs will not work correctly with multiple API instances.
- Mongo operations that update several documents do not use transactions.
- One shared MySQL connection rather than a pool.
- Initial MySQL failure leaves attendance polling stopped until the process restarts.

- Several core production modules disable TypeScript checking with `@ts-nocheck`.
- Large resolver files combine authorization, mapping, database access, analytics, and business logic.

Mobile App

Main weaknesses:

- A live-looking ExerciseDB/RapidAPI key is embedded in `Mobile App/app.json`. It should be considered exposed and rotated.
- The Clerk configuration is a test publishable key baked into the application.
- Development always uses the hosted API because `app.json` already contains an API URL, making the local LAN fallback code effectively dead.
- Global cleartext traffic is enabled on Android and arbitrary network loads are allowed on iOS.
- Expo updates are disabled, and the production Android build outputs an APK rather than an app bundle.
- Multiple components poll every two or three seconds, including some polling while drawers are closed.
- No tests.
- Several screens exceed 1,000 to 3,000 lines.
- No coherent offline/error-recovery strategy.
- API-generated notifications are not consumed.

Web

Main weaknesses:

- GraphQL code generation is broken after the merge.
- Duplicate GraphQL source/generated trees.
- Authentication tokens are persisted in localStorage.
- WebSocket authentication can start before Clerk is ready.
- Extensive polling overlaps with subscriptions.
- System notifications are feature-flagged off by default.
- PDF generation downloads fonts from GitHub at runtime.
- Large pages, some around 1,500 to 2,500 lines, combine data access, forms, tables, and exports.
- Only one real test exists, and it targets a simulation hook that does not appear to be part of the main application flow.
- The CI test command uses interactive `vitest` rather than explicit `vitest run`.
- An old ESLint report and `Attendance_backup.tsx` remain in the repository, while several important lint rules have been disabled.

Recommended revision order

Phase 1: Immediate containment

- Rotate the exposed RapidAPI key.
- Remove production access to development sign-in operations.
- Restrict protected fields in `updateUser`.
- Correct the known resolver authorization gaps.
- Isolate and reduce privileges for the attendance database.

Phase 2: Establish authoritative source

- Choose the active API tree.
- Remove or archive nested API copies outside production source.
- Select one Web GraphQL tree.
- Introduce a root workspace with documented commands and environment contracts.
- Preserve or reconstruct repository history where possible.

Phase 3: Unify core state machines

- Make membership activity derive from one authoritative service and transaction state.
- Choose either direct purchase or administrator-approved subscription.
- Define one authentication model.
- Define one notification model shared by Web and Mobile.

Phase 4: Repair contracts and delivery

- Point both clients at the active schema.
- Regenerate all GraphQL types and operations.
- Add a CI contract-drift check.
- Clean output directories before compiling.
- Add API and Mobile CI.
- Add environment validation and health/readiness endpoints.

Phase 5: Add safety tests

Prioritize tests for:

- Resolver authorization.
- Membership lifecycle and expiry.
- Subscription approval.
- Attendance mapping.
- Coach/member relationships.
- Session ownership.
- Clerk and legacy authentication integration.

Phase 6: Refactor and optimize

- Break up the largest files.
- Replace aggressive polling with focused refreshes or reliable subscriptions.
- Move scheduled jobs and PubSub to infrastructure that supports multiple instances.
- Bundle report fonts and other runtime assets locally.
- Formalize production Mobile release settings.
- Remove stale build artifacts and compatibility flags.

Review limitations

This was a static source and configuration review.

- Dependencies were not installed.
- Hosted services were not contacted.
- Builds and tests were not executed because dependencies were absent and the task was review-only.
- Dependency vulnerability/CVE auditing was not performed.
- Actual deployment environment variables and database contents were not inspected.

Those items should be covered in a separate verification phase after the critical containment work is planned.